Design a Simple Compiler Front-End using Lex & Yacc

CSM 322 — Theory of Compilers | Final Project Report

Student Name:	آسر محمد على خيرالله	Student ID:	23015780
Instructor:	Reham Hussein Gabr	Date:	April 5, 2026

1. Project Overview

This project implements a complete compiler front-end for a small interpreted programming language using **Flex** (lexical analysis) and **Bison** (syntax and semantic analysis). The compiler reads a source program, tokenizes it, parses it according to a formal grammar, performs semantic checks, evaluates statements, and optionally generates a three-address Intermediate Representation (IR) saved to `ir_output.txt`.

The project goes beyond the minimum requirements by implementing bonus features including `if` conditionals, `while` loops, IR generation, and an identifier-length semantic check — demonstrating a full pipeline from source text to execution.

2. Objective

The main objectives of this project are:

- Apply Lex/Flex to build a lexical analyzer that converts source text into tokens.
- Apply Yacc/Bison to build a parser that validates program structure using BNF grammar.
- Implement semantic analysis: symbol table management, variable lookup, and runtime evaluation.
- Detect and report semantic errors (undefined variables, division by zero, identifier length).
- Generate a three-address Intermediate Representation (IR) as a bonus deliverable.

3. Lexical Analysis — Lex / Flex

The file `project.l` defines the lexical rules using regular expressions. Flex reads the source program character by character and produces a stream of tokens for the parser.

Token Definitions

Token	Pattern / Rule	Description
`PRINT`	`"print"`	Print keyword
`IF`	`"if"`	Conditional keyword
`WHILE`	`"while"`	Loop keyword
`NUMBER`	`[0-9]+`	Integer literal (stored as int)
`IDENTIFIER`	`[a-zA-Z_][a-zA-Z0-9_]*`	Variable name (stored as string)
`GE, LE, EQ, NE`	`>=, <=, ==, !=`	Comparison operators

`NEWLINE`	`\n`	Statement separator
Whitespace	`[ \t\r]+`	Skipped (ignored)
Comments	`"//[^\n]*"`	Single-line comments, ignored
Other	`.`	Single character returned as-is (+, -, *, /, =, etc.)

The lexer uses `%option noyywrap yylineno` to track line numbers (used in syntax error messages) and to avoid requiring a wrap function at end-of-file.

4. Syntax Analysis — Yacc / Bison

The file `project.y` defines the grammar rules in BNF form. Bison generates an LALR(1) parser that consumes tokens from Flex and builds an Abstract Syntax Tree (AST) in memory.

Grammar Summary

- **program** → list of lines (statements separated by newline or semicolon)
- **statement** → assignment | print | if | while | block | expression
- **expression** → arithmetic (+, -, *, /), comparison (>, <, >=, <=, ==, !=), unary minus, parenthesized sub-expressions, number, identifier
- **block** → `{ statements }` — groups multiple statements as one body

Operator Precedence

Bison precedence rules (lowest to highest):

- Comparison: `==, !=, >, <, >=, <=`
- Additive: `+, -`
- Multiplicative: `*, /`
- Unary: `-` (right-associative, UMINUS)

The parser builds AST nodes using helper functions such as `make_assign()`, `make_binop()`, `make_if()`, and `make_while()`. All nodes are linked into a `StmtList` which represents the full program.

5. Semantic Analysis

5.1 Symbol Table

A fixed-size symbol table (array of up to 128 `Symbol` structs) stores variable names and their integer values at runtime. Each entry holds a name (max 63 characters) and a value. The functions `find_symbol()`, `get_symbol_value()`, and `set_symbol_value()` manage lookups and insertions.

5.2 Assignment Handling

When an assignment statement is executed, the right-hand expression is evaluated first. If no semantic error occurred during evaluation, the result is stored in the symbol table using `set_symbol_value()`. If the variable does not exist, a new entry is created; if it already exists, its value is updated.

5.3 Variable Lookup in Expressions

During expression evaluation, `get_symbol_value()` looks up the variable in the symbol table. If the variable is not found, a semantic error is reported: *"Semantic error: undefined variable 'x'"*, and the expression returns 0 to allow execution to continue and collect all errors.

6. Error Detection and Testing

Error Type	Trigger	Error Message
Undefined Variable	Using a variable that was never assigned	`Semantic error: undefined variable 'z'`
Division by Zero	Dividing any value by 0 at runtime	`Semantic error: division by zero`
Identifier Too Long	Variable name exceeds 63 characters	`Semantic error: identifier 'very_long...' exceeds max length 63`
Symbol Table Overflow	More than 128 distinct variables declared	`Semantic error: symbol table overflow (max 128 variables)`

All errors are reported via `report_semantic_error()` which increments a global counter. At the end, if any semantic errors occurred, *"Execution finished with N semantic error(s)."* is printed and the program exits with code 1. This allows errors to accumulate rather than stopping at the first one.

7. Bonus Features

7.1 if Statement

The `if (condition) statement` construct evaluates the condition expression. If the result is non-zero (true), the body statement is executed. The body can be a single statement or a block enclosed in braces.

7.2 while Loop

The `while (condition) statement` construct repeatedly evaluates the condition and executes the body as long as the condition is non-zero and no semantic error occurs during condition evaluation.

7.3 Intermediate Representation (IR) Generation

After a successful parse, the program generates a three-address IR into `ir_output.txt`. Temporary variables (t1, t2, ...) hold intermediate results, and labels (L1, L2, ...) are used for control flow in `if` and `while` statements.

Sample IR output (excerpt from `ir_output.txt`):

```
# Three-address intermediate representation
x = 5
y = 3
t1 = x + y
print t1
t2 = x + y
t3 = t2 * 2
print t3
...
t6 = x > y
ifFalse t6 goto L1
print x
L1:
L2:
t7 = y < 5
ifFalse t7 goto L3
t8 = y + 1
y = t8
```

```
goto L2
L3:
```

7.4 Identifier Length Validation

The function `validate_identifier_name()` checks that variable names do not exceed 63 characters (the symbol table entry size minus 1). If exceeded, a semantic error is reported and the operation is skipped.

8. Build and Run Steps

Run the following three commands inside the project directory:

Step 1 — Generate parser with Bison:

```
bison -d project.y
```

Step 2 — Generate lexer with Flex:

```
flex project.l
```

Step 3 — Compile all generated C files:

```
gcc project.tab.c lex.yy.c -o project -lfl
```

Step 4 — Run with an input file:

```
./project < test_all_cases.txt
```

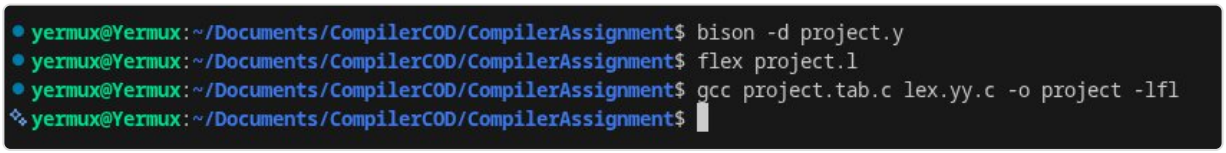

Figure 1: Terminal showing successful build commands (bison → flex → gcc)

9. Sample Runs

9.1 Test Input File

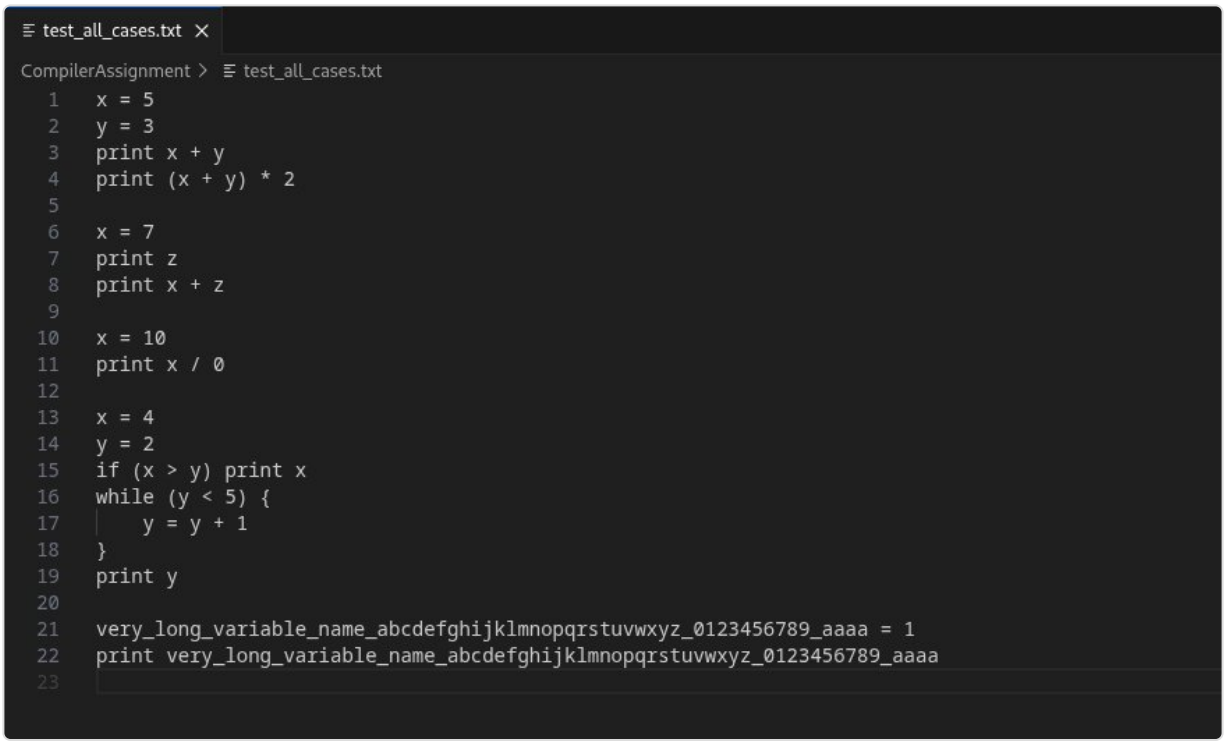

Figure 2: Complete test input file (test_all_cases.txt) covering all language features

The test file exercises every feature in sequence:

- Lines 1–4: Variable assignment and arithmetic (x=5, y=3, print x+y, print (x+y)*2)
- Lines 6–8: Use of undefined variable 'z' (triggers semantic error)
- Lines 10–11: Division by zero (x=10, print x/0)
- Lines 13–19: if statement and while loop with braces
- Lines 21–22: Identifier exceeding 63 characters

9.2 Program Output

```
yermux@Yermux:~/Documents/CompilerCOD/CompilerAssignment$ ./project < test_all_cases.txt
8
16
Semantic error: undefined variable 'z'
Semantic error: undefined variable 'z'
Semantic error: division by zero
4
5
Semantic error: identifier 'very_long_variable_name_abcdefghijklmnopqrstuvwxyz_0123456789_aaaa' exceeds max length 63
Semantic error: identifier 'very_long_variable_name_abcdefghijklmnopqrstuvwxyz_0123456789_aaaa' exceeds max length 63
Execution finished with 5 semantic error(s).
yermux@Yermux:~/Documents/CompilerCOD/CompilerAssignment$
```

Figure 3: Runtime output showing correct values, semantic errors, and final error summary

Output explanation:

- **8** — result of x + y = 5 + 3
- **16** — result of (x + y) * 2 = 8 * 2
- *Semantic error: undefined variable 'z'* — 'z' was used without assignment (appears twice)
- *Semantic error: division by zero* — print x / 0 with x = 10
- **4** — if (x > y) prints x = 4
- **5** — while loop increments y from 2 to 5, then prints y
- *Semantic error: identifier exceeds max length 63* — long variable name rejected
- *Execution finished with 5 semantic error(s)* — total error count

---

10. Conclusion

This project successfully implements a complete compiler front-end for a mini interpreted language, covering all three classic compiler phases: lexical analysis (Flex), syntax analysis (Bison/LALR(1)), and semantic analysis (symbol table + runtime evaluation). Beyond the required features, the project adds bonus capabilities including `if` statements, `while` loops, three-address IR generation, and an identifier-length semantic check.

The implementation demonstrates how theoretical compiler concepts — tokenization, parsing, AST construction, and semantic validation — translate into working C code. All test cases produced the expected results, and error messages are clear and informative, showing that the compiler correctly handles both valid and invalid programs.

11. GitHub Repository

The full source code (`project.l` and `project.y`) is publicly available at:
https://github.com/asserkhairallah2/CompilerFrontendAssignment

---

12. References

[1] Course Material. *LexAndYaccTutorial.pdf* — provided as a reference material attached with the assignment instructions for CSM 322: Theory of Compilers, Faculty of Information Systems & Computer Science, October 6 University.